



Introduction to Streams

☑ Stream:

A **stream** is a way to **handle reading or writing data** continuously, **piece by piece (chunks)**, instead of loading it all at once.

☑ Why Streams?

Efficient for **large files** or **real-time data** (e.g., video, network data) because it **reduces memory usage** and improves performance.

☑ Built-in Module:

Streams are part of Node.js **core modules** (like `fs`, `http`).

Types of Streams in Node.js

1. Readable Streams

Readable streams are used to **read data** from a source, such as reading files or receiving HTTP requests.

2. Writable Streams

Writable streams are used to **write data** to a destination, such as writing to a file or sending HTTP responses.

3. Duplex Streams

Duplex streams are streams that can **both read and write data**. These are useful for situations like network communication, where you both send and receive data.

4. Transform Streams

Transform streams are a special type of duplex stream where the **output is a transformation of the input**. They modify or process the data as it passes through the stream, such as compressing or encrypting data.



Readable Streams (Node.js)

◇ What is a Readable Stream?

- A **Readable Stream** is used to read data in chunks (piece by piece).
- Useful for **reading large files**, **handling network responses**, etc.
- **Streams are memory-efficient** compared to reading the whole file at once.

◇ Key Methods & Events

Method / Event	Description
----------------	-------------

Method / Event	Description
<code>.createReadStream()</code>	Creates a readable stream from a file.
<code>'data'</code>	Event emitted when a chunk of data is available.
<code>'end'</code>	Event emitted when no more data to read.
<code>'error'</code>	Event emitted on error while reading.
<code>highWaterMark</code>	Buffer size limit per chunk (in bytes).

◇ Example: Reading a file in chunks

```
const fs = require('fs');

const readStream = fs.createReadStream('chars.txt', {
  highWaterMark: 4 // reads 4 bytes at a time
});

readStream.on('data', (chunk) => {
  console.log('Chunk:', chunk.toString());
});

readStream.on('end', () => {
  console.log('Finished reading');
});

readStream.on('error', (err) => {
  console.error('Error:', err);
});
```

☑ `highWaterMark: 4` means it reads 4 bytes at a time.

◇ Real-life Use Cases

- Reading large files without loading all into memory.
- Streaming video/audio files.
- Receiving data from APIs in chunks.

◇ Extra: Write to another file while reading

```
const fs = require('fs');

const readStream = fs.createReadStream('chars.txt', { highWaterMark: 4 });
const writeStream = fs.createWriteStream('copy.txt');

readStream.on('data', (chunk) => {
  writeStream.write(chunk);
});
```

```
readStream.on('end', () => {  
  console.log('File copied');  
});
```

◇ Summary

- Streams = efficient data handling.
- Use `'data'`, `'end'`, and `'error'` events to control flow.
- `highWaterMark` helps manage chunk size.

Readable Stream States (Node.js)

◇ 1. Initial State

- The stream is **created but not started** yet.
- No data has been consumed or read.
- ◇ `readableFlowing: null`
- ◇ `readableEnded: false`
- ◇ `isPaused(): false`

◇ 2. Flowing State

- The stream is **actively reading** and pushing data using `'data'` events.
- Data flows automatically to the consumer.
- ◇ `readableFlowing: true`
- ◇ `readableEnded: false`
- ◇ `isPaused(): false`

◇ 3. Paused State

- The stream is **paused**, meaning it's not pushing data automatically.
- You can resume it using `.resume()`.
- ◇ `readableFlowing: false`
- ◇ `readableEnded: false`
- ◇ `isPaused(): true`

◇ 4. Ended State

- All data has been read, and `'end'` event emitted.
- Stream has finished.
- ◇ `readableFlowing: true`
- ◇ `readableEnded: true`
- ◇ `isPaused(): false`

Readable Stream States Summary Table

 State	readableFlowing	readableEnded	isPaused()
Initial	null	false	false
Flowing	true	false	false
Paused	false	false	true
Ended	true	true	false

Internal Buffer – Readable Streams (Node.js)

- Temporary storage inside the stream for incoming data
- Data stored until you read it
- Size controlled by **highWaterMark**
- If buffer full → stop reading from source
- If buffer empty → wait for new data
- Works in both flowing & paused modes

Writable Streams (Node.js)

- **Purpose:** Send/write data to a destination (e.g., file, network)
- **Main Methods:**
 - **write(chunk)** → send data
 - **end()** → finish writing
- **Internal Buffer:**
 - Temporary storage for outgoing data
 - Size controlled by **highWaterMark**
 - If full → **write()** returns **false** (wait for **'drain'**)
- **Events:**
 - **'drain'** → buffer ready for more data
 - **'finish'** → all data written
 - **'error'** → writing failed

Example

```
const fs = require('fs');

// Create writable stream to a file
const writeStream = fs.createWriteStream('output.txt');

// Write data
writeStream.write('Hello, ');
writeStream.write('this is a writable stream example.');
```

// End writing

```
writeStream.end();

// Events
writeStream.on('finish', () => {
  console.log('All data written successfully.');
```

Backpressure (Node.js Streams)

- **Meaning:** Happens when the **consumer** (reader/writer) is slower than the **producer**
 - **Cause:** Data builds up in the internal buffer
 - **Effect:** Memory usage increases, performance drops
 - **Writable Streams:** If `write()` returns `false` → backpressure is active
 - **Fix:** Wait for `'drain'` before writing again, or pause reading until buffer is free
-

Example – Handling Backpressure

```
const fs = require('fs');

const readable = fs.createReadStream('bigfile.txt'); // large file
const writable = fs.createWriteStream('copy.txt');

// Manual backpressure handling
readable.on('data', (chunk) => {
  const canWrite = writable.write(chunk);
  if (!canWrite) {
    // Pause reading until drain event
    readable.pause();
    writable.once('drain', () => readable.resume());
  }
});

readable.on('end', () => {
  writable.end();
  console.log('File copied successfully.');
```

Writable Stream States – Node.js

1. Writable (Initial)

- **Meaning:** Ready to accept data
- **Check:**

```
console.log(stream.writable); // true if writable
```

2. Corked

- **Meaning:** Data buffered, not sent until uncorked
- **Check:**

```
console.log(stream.writableCorked); // 0 = not corked, >0 = corked
```

3. Ended

- **Meaning:** `end()` called, no more writes allowed (data may still be flushing)
- **Check:**

```
console.log(stream.writableEnded);
```

4. Finished

- **Meaning:** All data flushed after `end()`
- **Check:**

```
console.log(stream.writableFinished);
```

Piping Streams – Node.js

- **What:** Connects output of one stream directly to input of another
- **Why:** Automatically handles data flow & backpressure
- **Syntax:**

```
source.pipe(destination)
```

- **Chainable:** Can pipe through multiple streams
- **Stops:** Destination closes when source ends (unless { `end: false` })

Example

```
const fs = require('fs');

fs.createReadStream('input.txt')
  .pipe(fs.createWriteStream('output.txt'))
  .on('finish', () => console.log('File copied successfully.'));
```

Piping Using `pipeline()` – Node.js

- **What:** A safer way to pipe streams
- **Why:** Automatically handles **backpressure** and **errors**
- **Import:**

```
const { pipeline } = require('stream');
```

- **Syntax:**

```
pipeline(source, destination, callback);
```

- **Callback:** Called when the pipeline **completes** or **fails**
- **Advantage:** Prevents unhandled errors unlike `pipe()`

Example

```
const fs = require('fs');
const { pipeline } = require('stream');

pipeline(
  fs.createReadStream('input.txt'),
  fs.createWriteStream('output.txt'),
  (err) => {
    if (err) console.error('Pipeline failed:', err);
    else console.log('Pipeline succeeded.');
  }
);
```

Duplex, Transform & PassThrough Streams – Node.js

1. Duplex Stream

- Can **read** and **write** data simultaneously
- **Example:** `net.Socket`

```
const { Duplex } = require('stream');
```

2. Transform Stream

- A type of **Duplex** stream that **modifies** data while reading/writing
- **Example:** `zlib.createGzip()`

```
const { Transform } = require('stream');
```

3. PassThrough Stream

- A **Transform** stream that **does not modify** data
- Useful for debugging or monitoring

```
const { PassThrough } = require('stream');
```

Data Streams

◇ Types of Data Streams

When a process starts, it has **3 default data streams**:

- **stdin** → Standard Input → *used to read data*
- **stdout** → Standard Output → *used to write data*
- **stderr** → Standard Error → *used to write errors & warnings*

☑ **All are duplex streams** → can **read** and **write**.

◇ Accessing Streams

- **Readable Stream** → `process.stdin`
- **Writable Streams** → `process.stdout`, `process.stderr`

◇ Important Points

- `console.log()` → internally uses **stdout**.
- **stderr** → works like stdout but is used **only for errors**.
- These streams are available through the **process object**.

◇ File Descriptors (fd)

Stream	File Descriptor
stdin	0
stdout	1
stderr	2

Piping & Redirection of Data Streams

◇ Piping of Data Streams

- **Piping** connects the **stdout** of one process to the **stdin** of another.
- Works only on **Linux/WSL**.
- **Pipe operator:** |

Example 1

```
echo hii | node index.js
```

→ The **stdout** of `echo hii` is piped to `index.js` **stdin**.

Example 2

```
node script.js | node index.js
```

→ The **stdout** of `script.js` is piped to `index.js` **stdin**.

⚡ Important:

- | only pipes **stdout** → **stdin**.
- **stderr** is **not** piped automatically.

◇ Redirection of Data Streams

- **Redirection** sends the **stdout** or **stderr** of a process to a **file** instead of the terminal.

Redirect stdout to a file

```
node script.js > command.txt
```

☑ Saves **stdout** of `script.js` into `command.txt`.

Redirect stderr to a file

```
node script.js 2> command.txt
```

- ☑ Saves **stderr** of `script.js` into `command.txt`.

Note: `2` → file descriptor of **stderr**.

Redirect both stdout & stderr

```
node script.js > command.txt 2>> command.txt
```

- **stdout** → written to `command.txt`.
- **stderr** → **appended** to `command.txt`.

Read data from a file into stdin

```
node index.js < command.txt
```

- 📁 Reads **data** from `command.txt` and sends it to `index.js` **stdin**.

⚡ Why Streams Are Fast

- 📦 **Chunk-based processing** → Data is handled in **small pieces**, not all at once.
- ⚡ **Memory efficient** → Uses less RAM since the full data isn't loaded.
- 🔄 **Continuous flow** → Starts processing **before** receiving all data.
- ⚙️ **Non-blocking I/O** → Works **asynchronously**, avoiding delays.

- ☑ Streams = **Faster + Efficient + Scalable**

📁 File Descriptor in Node.js

◇ What is a File Descriptor?

- A **file descriptor** is a **non-negative integer (0 or positive)**.
- It acts like an **address/reference** to an **open file** in the operating system.

◇ Example Usage

Asynchronous (Callback)

```
fs.open(path, (err, fd) => {  
  console.log(fd); // Prints file descriptor  
});
```

Synchronous

```
const fd1 = fs.openSync(path);  
console.log(fd1); // Prints file descriptor
```

◇ Important Points

- Each **open file** gets a **unique file descriptor**.
- In Node.js, descriptors **start from 3** because:

Stream	File Descriptor
stdin	0
stdout	1
stderr	2

- The **next opened file** gets **3**, then **4**, and so on.

Reading a File Using File Descriptor in Node.js

◇ Reading a File with `fs.read()`

The `fs.read()` method reads data from a file **using its file descriptor (fd)**.

Basic Syntax

```
fs.read(fd, (err, bytesRead, bufferData) => {  
  console.log(err);           // Any error, if occurs  
  console.log(bytesRead);     // Number of bytes read  
  console.log(bufferData);    // Data read into buffer  
  console.log(bufferData.length); // Default buffer size = 16 KB  
});
```

◇ Reading with Custom Buffer

You can specify a **custom buffer size** using the `buffer` option.

```
fs.read(fd, { buffer: Buffer.alloc(10) },
  (err, bytesRead, bufferData) => {
    console.log(err);
    console.log(bytesRead);    // Bytes read
    console.log(bufferData);    // Data inside custom buffer
    console.log(bufferData.byteLength);
  }
);
```

☑ Here, a **10-byte buffer** is allocated instead of the default **16 KB**.

◇ Additional Options

- **position** → Starting point in the file to begin reading.
- **length** → Number of bytes to read from the given position.

Opening Files in Different Modes in Node.js

◇ Opening a File

In Node.js, you can open files using:

```
const fd = fs.openSync(path, mode);
```

- **path** → File path
- **mode** → How the file should be opened

◇ Default Mode

- If no mode is provided, **files open in read mode (r)** by default.

◇ File Open Modes

Mode	Description
r	Open file for reading . (Error if file doesn't exist)
w	Open file for writing . (Creates file if not exists, overwrites if exists)
a	Open file for appending . (Creates file if not exists)
mode+	Open file in the specified mode plus read/write support

◇ Examples

```
// Open in read mode
const fd1 = fs.openSync("file.txt", "r");

// Open in write mode
const fd2 = fs.openSync("file.txt", "w");

// Open in append mode
const fd3 = fs.openSync("file.txt", "a");

// Open in read + write mode
const fd4 = fs.openSync("file.txt", "r+");
```

Writing to a File Using File Descriptor in Node.js

◇ Opening the File

Before writing, **make sure** the file is opened in **write mode**:

```
const fd = fs.openSync(path, "w");
```

◇ Writing Data (Asynchronous)

Use **fs.write()** to write data into a file:

```
fs.write(fd, content, (err, bytesWritten, str) => {
  console.log(err);           // Any error, if occurs
  console.log(bytesWritten);  // Number of bytes written
  console.log(str);           // Written string content
});
```

- **fd** → File descriptor
- **content** → Data to write
- **bytesWritten** → Number of bytes successfully written
- **str** → Written data

◇ Writing Data (Synchronous)

Use **fs.writeSync()** for synchronous writing:

```
const bytesWritten = fs.writeSync(fd, content);
console.log(bytesWritten); // Number of bytes written
```

Handling Files Using Promises (Node.js)

1. Import Module

```
import fs from "fs/promises";
```

Use the **fs/promises** module to work with files using **async/await**.

2. Open a File

```
const fileHandle = await fs.open("time.txt", mode);
```

- **fs.open(path, mode)** → Opens a file.
- **Returns:** `fileHandle` object with methods & properties.

Mode Examples:

Mode	Description
<code>r</code>	Read only
<code>w</code>	Write only (creates/overwrites file)
<code>a</code>	Append mode

3. Read from a File

```
const r = await fileHandle.read(options);
```

- **Returns:** `{ bytesRead, buffer }`
- **Options** (*optional*):
 - `{ length: n }` → Number of bytes to read.
 - `{ position: pos }` → Start reading from a specific position.

4. Write to a File

```
const w = await fileHandle.write(content, options);
```

- **Returns:** `{ bytesWritten, buffer }`
- **Parameters:**
 - **content** → Data you want to write.
 - **Options** (*optional*):

- `{ position: pos }` → Start writing from a specific position.

5. Close the File

```
await fileHandle.close();
```

Always close the file after operations to prevent **memory leaks**.

Summary Table

Operation	Method	Returns
Open file	<code>fs.open(path, mode)</code>	<code>fileHandle</code> object
Read file	<code>fileHandle.read()</code>	<code>{ bytesRead, buffer }</code>
Write file	<code>fileHandle.write()</code>	<code>{ bytesWritten, buffer }</code>
Close file	<code>fileHandle.close()</code>	<code>undefined</code>

Working with Streams Using Promises in Node.js

1. Import the fs/promises Module

```
import fs from "fs/promises";
```

fs/promises → Provides Promise-based file system methods.

2. Open Files

```
const fileHandle1 = await fs.open("time.txt"); // Read mode (default)
const fileHandle2 = await fs.open("abc.txt", "w+"); // Read & write mode
```

fs.open(path, mode) → Opens a file and returns a file handle.

Modes:

- **"r"** → Read only
- **"w"** → Write only (*creates/overwrites*)
- **"w+"** → Read & write (*creates/overwrites*)

3. Create Readable and Writable Streams

```
const readStream = fileHandle1.createReadStream();
const writeStream = fileHandle2.createWriteStream();
```

- **createReadStream()** → Reads data in chunks.
- **createWriteStream()** → Writes data in chunks.

Same as normal streams but using Promise-based file handles.

4. Pipe Data from One File to Another

```
readStream.pipe(writeStream);
```

pipe() → Sends data from the readable stream directly into the writable stream.

5. Close File Handles

```
writeStream.on("close", async () => {
  await fileHandle1.close();
  await fileHandle2.close();
  console.log("file Closed");
});
```

- Listen for **"close"** event → Ensures streams are fully done.
- Always close file handles to free system resources.

◇ Summary Table

Concept	Method / Event	Purpose
Open file	<code>fs.open(path, mode)</code>	Returns file handle
Create read stream	<code>.createReadStream()</code>	Read file in chunks
Create write stream	<code>.createWriteStream()</code>	Write file in chunks
Pipe data	<code>readStream.pipe()</code>	Transfer data efficiently
Close file	<code>.close()</code>	Close file handle
Event listener	<code>"close"</code>	Triggered when stream closes